

Rapport TP1 RMI

HAI704I : Architectures logicielles distribuées



Table des matières

Introduction.....	2
A0. Mise en route et vérification de l'environnement.....	3
A1. Un premier objet distant : l'animal.....	3
A2. L'espèce : une donnée qui traverse le réseau.....	4
A3. Le dossier de suivi : à vous de décider.....	4
A4. Le cabinet vétérinaire : un point d'entrée unique.....	5
A5. Créer un patient depuis le client.....	5
A6. Alertes : le serveur rappelle ses clients.....	5
A7. Ce qui traverse vraiment le réseau.....	6
A8. Une application utilisable : l'interface en ligne de commande.....	6
Synthèse.....	8
Regard critique.....	8
Usage des outils d'intelligence artificielle générative.....	8
Repo Github.....	9

Introduction

Ce TP a pour objectif de découvrir le fonctionnement de Java RMI à travers la réalisation d'une application distribuée de cabinet vétérinaire. Le travail consiste à mettre en place une communication entre un client et un serveur, à manipuler des objets distants et sérialisables, puis à ajouter des fonctionnalités comme la gestion des patients, des dossiers de suivi et des alertes.

A0. Mise en route et vérification de l'environnement

Le projet a été développé avec OpenJDK 25.

J'ai ensuite testé les exemples RMI fournis afin de vérifier que la communication client-serveur et le registre RMI fonctionnaient correctement.

Enfin, j'ai créé la structure de l'application avec trois projets séparés : common, server et client. Le projet common regroupe les éléments partagés entre le client et le serveur, tandis que les deux autres projets contiennent chacun leur partie spécifique.

A1. Un premier objet distant : l'animal

Pour cette première étape, j'ai créé l'interface `Animal` dans le projet common. Elle étend `Remote` et les méthodes accessibles à distance déclarent `RemoteException`. Cela permet au client et au serveur de partager le même contrat sans que le client ait besoin de connaître l'implémentation réelle.

L'implémentation `AnimalImpl` se trouve uniquement côté serveur. Elle étend `UnicastRemoteObject`, ce qui permet à l'objet d'être exporté et donc accessible à distance.

Le serveur crée ensuite un objet animal et le rend accessible via le registre RMI. Côté client, le registre est récupéré avec `LocateRegistry.getRegistry()`, puis l'objet distant est obtenu avec `lookup()`.

J'ai validé cette étape en lançant le serveur et le client dans deux processus séparés et en appelant les méthodes de l'animal depuis le client.

Interface `Animal` dans common :

```
public interface Animal extends Remote {  
    String getNomAnimal() throws RemoteException;  
}
```

A2. L'espèce : une donnée qui traverse le réseau

Dans cette étape, j'ai remplacé l'attribut représentant l'espèce par une classe `Espec`. Cette classe est placée dans le projet commun, car elle doit être connue à la fois par le client et par le serveur.

J'ai choisi de rendre `Espec` sérialisable en implémentant `Serializable`. Lorsqu'un client récupère l'espèce d'un animal avec `getEspec()`, il reçoit donc une copie de l'objet et non une référence vers l'objet conservé sur le serveur.

Pour vérifier ce comportement, j'ai récupéré l'espèce côté client puis modifié cette copie locale. En demandant de nouveau l'espèce au serveur, sa valeur d'origine était toujours présente. Les valeurs obtenues avec `System.identityHashCode()` étaient également différentes entre le client et le serveur, ce qui confirme qu'il s'agit de deux objets Java distincts.

Extrait de la classe `Espec` dans `common` :

```
public class Espec implements Serializable {  
    private String nom;  
    private String esperanceVie;  
}
```

A3. Le dossier de suivi : à vous de décider

Chaque animal possède un dossier de suivi contenant son état de santé ainsi qu'une liste d'observations. Pour cette partie, j'ai choisi de faire de `DossierSuivi` un objet distant plutôt qu'un objet sérialisable.

Ce choix permet de conserver un seul dossier côté serveur. Ainsi, lorsqu'un client modifie l'état de santé ou ajoute une observation, la modification est directement appliquée sur le dossier partagé et peut être vue par les autres clients.

L'interface `DossierSuivi` étend donc `Remote`, tandis que `DossierSuiviImpl` est implémentée côté serveur et étend `UnicastRemoteObject`. Les opérations de consultation et de modification du dossier sont réalisées à distance.

Pour valider ce choix, j'ai modifié le dossier depuis un client puis je l'ai consulté à nouveau. La modification était toujours présente, ce qui montre que le dossier n'est pas une copie locale mais bien un objet partagé conservé sur le serveur.

Extrait de l'interface `DossierSuivi` :

```
public interface DossierSuivi extends Remote {  
    String getEtatSante() throws RemoteException;  
    void setEtatSante(String etatSante) throws RemoteException;  
}
```

A4. Le cabinet vétérinaire : un point d'entrée unique

Pour éviter de publier chaque animal séparément dans le registre RMI, j'ai ajouté un objet distant `Cabinet` qui sert de point d'entrée unique vers les patients. Il permet notamment de récupérer la liste des animaux et d'en rechercher un à partir de son nom.

Le serveur publie uniquement le cabinet dans le registre sous le nom `Cabinet123`. Le client récupère ce cabinet puis accède aux animaux à travers lui. Comme `Animal` est lui-même un objet distant, les éléments récupérés dans la liste permettent toujours d'appeler les animaux à distance.

Lorsqu'aucun patient ne correspond au nom recherché, ma méthode `getPatientByName` renvoie `null`.

A5. Créer un patient depuis le client

J'ai ajouté au cabinet une méthode distante permettant d'enregistrer un nouveau patient depuis le client. Les informations nécessaires, comme le nom, le maître, la race, l'espèce et l'état de santé initial, sont envoyées au serveur.

L'espèce est transmise sous la forme d'un objet `Espèce` sérialisable, tandis que le nouvel `Animal` et son `DossierSuivi` sont créés directement côté serveur afin qu'ils restent des objets distants accessibles par les autres clients.

Pour vérifier le fonctionnement, j'ai ajouté un patient depuis le client puis contrôlé qu'il apparaissait bien dans la liste et qu'il pouvait être retrouvé par son nom.

A6. Alertes : le serveur rappelle ses clients

Pour mettre en place les alertes, j'ai utilisé le principe du patron `Observer`. L'interface `ObservateurConnexions` est définie dans le projet commun puis implémentée côté client par `ObservateurClient`. Cet objet est exporté afin que le serveur puisse appeler directement le client lorsqu'une alerte doit être envoyée.

Le cabinet permet ensuite aux clients de s'abonner ou de se désabonner des alertes. Lorsqu'un seuil de patients est franchi, le serveur parcourt la liste des observateurs abonnés et leur envoie une notification. Le client affiche alors le message reçu directement dans sa console. Ce fonctionnement correspond à un callback du client par le serveur, le client devient lui-même hébergeur d'un objet distant.

J'ai également pris en compte le cas où un client abonné n'est plus joignable. Si l'appel distant vers cet observateur échoue, l'exception est interceptée et l'observateur est retiré de la liste. Cela permet aux autres clients abonnés de continuer à recevoir les alertes normalement.

Le fonctionnement a été testé avec deux clients abonnés simultanément afin de vérifier qu'une même alerte est bien reçue dans les deux consoles et que la fermeture d'un client ne bloque pas les notifications destinées à l'autre.

Dans l'état actuel de l'application, les seuils sont franchis uniquement à la hausse, car aucune fonctionnalité de suppression de patient n'a été ajoutée. Un franchissement à la baisse reposerait sur le même principe.

A7. Ce qui traverse vraiment le réseau

Pour cette étape, j'ai créé une classe Chien héritant de Espece, mais uniquement dans le projet client. J'ai ensuite essayé d'envoyer une instance de cette classe au serveur lors de l'ajout d'un patient.

L'appel a échoué avec une `ClassNotFoundException`, encapsulée dans une `UnmarshalException` puis une `ServerException`. Le serveur recevait bien les données sérialisées, mais il ne connaissait pas la classe Chien et ne pouvait donc pas reconstruire l'objet.

Cette expérience montre qu'un flux sérialisé contient l'état de l'objet ainsi que le nom de sa classe, mais pas la définition de cette classe ni son bytecode. Le destinataire doit donc déjà disposer de la classe correspondante.

Pour corriger le problème, j'ai déplacé Chien dans le projet common, afin que le client et le serveur connaissent tous les deux cette classe. Après cette modification, l'ajout du patient fonctionne normalement.

A8. Une application utilisable : l'interface en ligne de commande

Pour rendre l'application directement utilisable, j'ai ajouté une interface en ligne de commande. Elle permet de lister et rechercher les patients, d'enregistrer un nouveau patient, de consulter et modifier son dossier de suivi, ainsi que de s'abonner ou se désabonner des alertes.

Le client est séparé en trois parties. La classe Client sert de point d'entrée : elle récupère l'hôte et le port, se connecte au registre RMI et obtient le cabinet. `InterfaceCLI` gère uniquement le menu, les saisies de l'utilisateur et l'affichage. Enfin, `LogiqueCLI` regroupe les opérations du client et les appels sur les objets distants. Cette organisation permet de garder la partie RMI en dehors de l'interface console, comme demandé dans le sujet.

Les saisies incorrectes sont traitées sans arrêter l'application. Un choix qui n'est pas un nombre ou un champ obligatoire laissé vide entraîne simplement l'affichage d'un message et une nouvelle saisie. De la même manière, lorsqu'un appel distant échoue, un message compréhensible comme « Impossible de contacter le serveur » est affiché à l'utilisateur.

J'ai également prévu un arrêt propre du client. Lorsqu'il quitte l'application alors qu'il est abonné aux alertes, le client tente de se désabonner puis désexporte son observateur local. Cela permet à la JVM de réellement se terminer, même si le serveur n'est plus accessible. C'est un point particulier de RMI puisqu'un objet distant encore exporté peut maintenir le processus en vie.

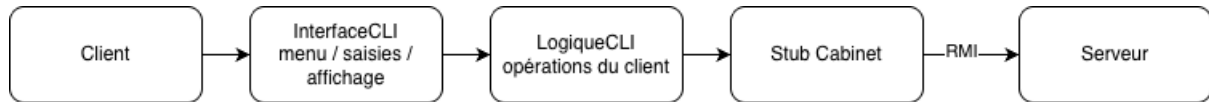


Figure - Séparation des responsabilités dans le client

La validation a été réalisée avec deux clients lancés dans des terminaux séparés. Une modification du dossier effectuée depuis un client est visible depuis l'autre, les alertes peuvent être reçues par les deux clients, une saisie invalide ne provoque pas d'exception et l'arrêt du serveur produit un message d'erreur sans faire quitter le client.

Synthèse

Ce TP m'a permis de mettre en place progressivement une application distribuée complète avec Java RMI. L'application finale permet à plusieurs clients de se connecter à un même cabinet vétérinaire, de consulter et ajouter des patients, de modifier leurs dossiers de suivi et de recevoir des alertes envoyées par le serveur.

Les différentes étapes m'ont permis de bien comprendre la différence entre un objet distant, et un objet sérialisable transmis par valeur. La principale difficulté a été la gestion des interactions dans les deux sens entre le client et le serveur, notamment pour les alertes. Il a fallu exporter un objet côté client, gérer les clients devenus injoignables et prévoir un arrêt propre afin qu'un observateur RMI encore exporté ne maintienne pas le processus actif.

L'application répond ainsi aux principales fonctionnalités demandées et peut être utilisée depuis plusieurs clients en parallèle. Une amélioration reste possible concernant les alertes lors du franchissement d'un seuil à la baisse : l'application ne propose actuellement aucune opération permettant de supprimer un patient et donc de faire diminuer leur nombre.

Regard critique

Même si l'application répond aux fonctionnalités demandées, plusieurs améliorations pourraient être apportées. La première serait d'ajouter des tests unitaires et des tests d'intégration afin de vérifier automatiquement les principales fonctionnalités et de mieux détecter les régressions lors de futures modifications.

L'ergonomie pourrait également être améliorée en ajoutant une interface graphique. La séparation actuelle entre InterfaceCLI et LogiqueCLI faciliterait cette évolution, puisque la nouvelle interface pourrait réutiliser la logique client existante sans avoir à réimplémenter les appels RMI.

Enfin, le projet pourrait être davantage structuré pour faciliter son évolution, notamment en réduisant certains traitements répétitifs et en renforçant la séparation entre la gestion des résultats, l'affichage et la logique métier.

Usage des outils d'intelligence artificielle générative

J'ai utilisé un outil d'intelligence artificielle principalement comme aide à la compréhension de certaines notions, notamment le patron Observer et son utilisation avec RMI.

Il m'a également servi à faciliter certains tests et à éviter de répéter manuellement certaines vérifications.

Repo Github

https://github.com/lohanSR/HAI704I_TP1